DOCUMENT RESUME

ED 284 452 FL 016 858

AUTHOR Underwood, John H.
TITLE Artificial Intelligence and CALL.
PUB DATE 82
NOTE 30p.; In: Smith, William Flint, Ed. Modern Media in Foreign Language Education: Theory and Implementation. The ACTFL Foreign Language Education Series; see FL 016 850.
PUB TYPE Information Analyses (070)

EDRS PRICE MF01 Plus Postage. PC Not Available from EDRS.
DESCRIPTORS *Artificial Intelligence; Communicative Competence (Languages); *Computer Assisted Instruction; *Computer Software; Educational Technology; Experimental Programs; Grammar; Individualized Instruction; *Language Processing; Programing Languages; *Second Language Instruction

ABSTRACT

The potential application of artificial intelligence (AI) to computer-assisted language learning (CALL) is explored. Two areas of AI that hold particular interest to those who deal with language meaning--knowledge representation and expert systems, and natural-language processing--are described and examples of each are presented. AI contribution to computers and natural language in such areas as understanding natural language, mechanical parsing, machine translation, speech recognition, and speech synthesis are outlined. Examples of CALL programs using AI are presented, including ELIZA, SHRDLU, LIESL, FAMILIA, JOHN AND MARY, JUEGOS COMUNICATIVOS, and STATION. Current major projects using AI in CALL are also described, including Project Athena, at the Massachusetts Institute of Technology; TALK (Teacher's Authoring Language Kit) Project, at the University of Rochester in New York; Project Alexis, at Mills College in California; and Intelligent Grammar Tutor, at the University of Pittsburgh in Pennsylvania. (CB)

****
* Reproductions supplied by EDRS are the best that can be made *
* from the original document. *
****

U.S. DEPARTMENT OF EDUCATION
Office of Educational Research and Improvement
EDUCATIONAL RESOURCES INFORMATION CENTER (ERIC)

☑ This document has been reproduced as received from the person or organization originating it.

☐ Minor changes have been made to improve reproduction quality.

• Points of view or opinions stated in this document do not necessarily represent official OERI position or policy.

"PERMISSION TO REPRODUCE THIS MATERIAL IN MICROFICHE ONLY HAS BEEN GRANTED BY

L. Fiddle

TO THE EDUCATIONAL RESOURCES INFORMATION CENTER (ERIC)."

8

ED284452

Artificial Intelligence and CALL

John H. Underwood
Mills College

Artificial Intelligence and Natural Language

Artificial Intelligence: An Overview

In using computers for language practice, one runs the risk of merely mechanizing the least attractive aspects of what we already do. The "wrong—try-again" approach, in which the computer asks all the questions, knows all the answers, and tosses students an occasional verbal reward to keep them going, will probably not go far toward helping them achieve real proficiency in the language. What is needed is practice that more closely resembles natural contextualized use of the language and that makes fuller application of the computer's capability for flexible and personalized interaction. It can be done. Researchers in the branch of computer science known as "artificial intelligence" (AI) have for years been experimenting with just such ways of using computers—flexible, natural, humanlike responses to human input.

Artificial intelligence is not without its controversy, starting with the very name by which it is known. It is doubtful that non–computer scientists would have paid much attention to the term coined by McCarthy thirty years ago if instead of "artificial intelligence" he had used "complex symbolic processing" or "heuristic programming." But McCarthy's

FL016855

John H. Underwood (Ph.D., University of California, Los Angeles) is Assistant Professor of Hispanic Studies and Linguistics at Mills College in Oakland, California. He is a member of ACTFL, CALICO, MLA, and TESOL, author of the widely acclaimed *Linguistics, Computers, and the Language Teacher: A Communicative Approach* (Newbury House) for which he won the Kenneth W. Mildenberger Award in 1984, and co-author (with Richard Bassein) of Juegos Comunicativos published by Random House.

BEST COPY AVAILABLE

catchy phrase stirs up passions. In one (extreme) view, artificial intelligence is nothing less than a contradiction in terms: intelligence is uniquely human. On the other side are the optimistic AI researchers who argue that intelligent behavior is independent of the medium in which it occurs: "Neurons or silicon—the difference is irrelevant" (Patrick Winston, quoted in Marback, 12).

Although the first image that comes to mind when one hears of "intelligent machines" may be something akin to R2-D2, robotics is actually only a small part of AI work. Artificial intelligence ranges from programs that play chess or prove mathematical theorems to those that aid in the diagnosis of mainframe computer failure or help prospectors find new oil deposits. The key in each case is that the computer is programmed to do things that would require intelligence if accomplished by a human. Artificial intelligence is thus a science somewhere on the border between psychology and computer programming, dedicated to developing techniques for solving intellectual problems. Some scientists like to think of AI as "scuba gear for the mind—it enables the mind to go places it hasn't been before" (Feigenbaum and McCorduck, 6, p. 44).

There are others who define AI in a slightly narrower way, using the term "intelligent" to refer only to those applications in which the program can be said to either draw inferences or to "learn" from the input it receives. Such programs are considered truly "intelligent" because they can internalize new knowledge and formulate conclusions not programmed into them. For the purposes of this chapter, however, the term artificial intelligence will encompass programs that range from the simple application of stored rules to simulated intelligent behavior. Topics not included are the murky—though admittedly fascinating—philosophical questions such as "Can machines think?" and "What is intelligence?"; rather, the principal focus is on the application, or potential application, of AI to computer-assisted language-learning (CALL) programs, and on guidelines for the creation of some clever, indeed even "intelligent," tools for language practice.

Knowledge Representation and Expert Systems

Two areas of AI are of particular interest to those who deal with language and meaning: knowledge representation and natural-language processing. Knowledge representation refers to the challenge of encoding in machine-readable form the kind of knowledge that is needed to solve problems, which can be very complex (the design of semiconductor chips) or seemingly trivial and commonsense problems (how to lead a robot to find its way around a coffee table). Much of current AI work is involved with coding this kind of knowledge in order to create so-called "expert systems." To accomplish this coding, a team of systems designers first studies a human

expert's process of reasoning through some narrowly defined problem, then programs this process into the system as a set of "if-then" rules. For example, in designing MYCIN, a Stanford system that can diagnose bacterial infections, doctors were consulted and their collective expertise was programmed into the system. The program is then able to reproduce the steps a doctor would go through in the process of analyzing symptomatic data and suggest a probable diagnosis. In one test of the system, MYCIN outperformed all of the medical experts evaluating it (Winograd, 27).

A project at the University of California (Berkeley) offers a good example of both the direction of current work on these systems and attendant problems. Known as the UNIX Consultant, the intent is to provide an on-line consulting service for users who have problems with UNIX, the principal operating system at UC Berkeley (and many other academic institutions). Once this expert system is operational, users' queries will be written in ordinary English to be processed by several different modules which, in turn, will parse the input (i.e., analyze its grammatical structure; see below) and symbolize its meaning in a manner that could then be interpreted by the knowledge base, a storehouse of information about UNIX. When a response is formulated, another program will generate an answer in English to represent this information to the user.

Key problems in the project have been concerned largely with meaning: how to represent a great deal of technical knowledge about UNIX, how to determine the meaning of the user's input sentence and infer from it the user's actual intent, plus the additional problem of generating an answer to the question in an intelligible manner. Another type of problem—the need for a wide and varied lexicon—was revealed when the Consultant was demonstrated to a group of UNIX users not familiar with the project and who used terms the system hadn't been programmed to expect. Those working on the project concluded that it may have been "overambitious" in scope.

Natural-Language Processing

Natural-language processing (NLP) involves all those areas in which the computer is programmed to deal with natural (human) language as meaningful symbols rather than just strings of characters. Unlike word processing, where the computer knows the text only as letters, NLP seeks to find ways for computers to "understand" language, albeit in some limited sense. This includes (1) work on "natural-language interfaces" or "natural-language front ends," which allow users to communicate with computers in ordinary English (or French, etc.); (2) machine translation, which attempts to understand the text enough to come up with an equivalent version rather than just a word-for-word mish-mash; (3) the particularly thorny problem of understanding speech, which requires

identifying the salient acoustic features of the human voice. Computer experts now often refer to NLP efforts as "amateur systems," since, unlike expert systems, NLP programs are designed to be used by laymen. Although much progress has been made in this area in the last few years, the real breakthroughs in NLP have yet to appear; the Japanese push to be first with the "Fifth Generation" supercomputers, may—some say—be the eventual catalyst for genuine advances.

Computers and Natural Language

Understanding Natural Language

There are basically two ways in which humans can communicate with a computer: (1) through a set of written instructions in a programming language such as BASIC or Pascal and (2) through an existing program, which serves as an interpreter—providing one follows its rules ("Enter name here: "). (A third possibility for the technically minded is to write the instructions in the computer's own language, known as machine language—a string of ones and zeros.) Clearly there would be many advantages if, as an alternative, one could give instructions to the computer in everyday English or some other natural language. The problems, however, are not straightforward. Human language is highly complex, redundant, and more often than not ambiguous. Computers, on the other hand, are basically symbol manipulators—the only thing they "know" is that a given symbol is or is not the same as another. A very complicated and cleverly designed program is required to enable a computer even to approximate an understanding of natural language.

Consider the problem this way: It is a fairly trivial task to program a computer to recognize written words (spoken words are another matter entirely), since a word, after all, is just a string of characters. Each of these words can have associated with it certain information that enables the system to "know" what it means, a sort of dictionary in which it can look up definitions. Less trivial, though still manageable, is to parse the sentence in which the word is found, i.e., analyze its structure and perhaps identify the parts of speech. But human understanding is many times more complex than word recognition and parsing sentence structure. Consider, for example, the kinds of knowledge needed to figure out the antecedent of *it* in "She dropped the plate on the table and broke it." Human listeners can look for a clue in the attendant discourse, perhaps something like "Remember that glass coffee table I told you about?" Context will also help straighten out cases of simple lexical ambiguity, such as "She cannot bear children" or "The rabbi married my sister." Likewise, idiomatic

expressions such as "run up" as in "He ran up a big bill" can be distinguished from the literal "He ran up a big hill" only because of our knowledge of the real world and the kinds of things one can truly "run up." In summary, natural language is "context-sensitive," i.e., meanings of words or phrases are sensitive to the context in which they appear. Conversely, programming languages are "context-free": in BASIC, for example, "run" is always a verb and it always means the same thing: "start the program."

In addition to what is known about a natural language and the world it symbolizes, part of everyday, commonsense knowledge deals with the way conversation works. This enables one to infer, for example, a logical relationship between two otherwise unconnected utterances:

Student 1: Did he say anything about the exam?

Student 2: I just got here.

Humans know the rules of conversation and assume a connection rather than a non sequitur. In the above case, common sense predicts a relationship between one's having just arrived in a class and an awareness of what had been said before entering. Speakers are normally not aware how much of what is understood depends on just such implicit connections.

One of the more humbling discoveries in natural-language processing is that language cannot be separated from pragmatic knowledge associated with common sense. The result is a curious paradox with respect to the success of natural-language programs. Machines do much better at tasks that require large amounts of highly specialized knowledge than at simple, everyday tasks: "the more highly structured knowledge is, the easier it is for us to codify it for computer use. On the other hand, getting around in the real world is not a highly structured task—the average house pet can manage it easily, but machines cannot" (Feigenbaum and McCorduck, 6, p. 59).

A complete system to understand language, even if the problem of recognizing speech sounds is ignored, then, must be capable of all of the following procedures: (1) lexical analysis (matching words to entries in the system's dictionary), (2) morphological analysis ("peeling off" and identifying affixes that may appear on words), (3) syntactic analysis (identifying parts of speech, phrase grouping, and overall structure), (4) semantic analysis (the dictionary meaning of the words) and pragmatic analysis (what the words mean in this context), and (5) enough real-world knowledge to make some sense of what is going on. The pitfalls are many, and subtle. The outlook, according to Winograd (27), is guarded: "software that mimics the full human understanding of language is simply not in prospect" (p. 144).

The following sections contain a discussion of subcomponents of natural-language processing of particular interest to foreign language

teachers: parsing, machine translation, speech recognition, and speech synthesis.

Mechanical Parsing

Parsing a sentence, mechanically or otherwise, means identifying its structure, a process that on a computer requires storing the words according to their syntactic categories (parts of speech) and specifying rules for forming possible sentences. The parser then proceeds word by word through the sentence, identifying the syntactic category of each while checking sequences of categories against those allowed by the rules, and assigning appropriate phrase structures to the sequences.

Suppose, for example, the input sentence is "The boy ate the pizza." The program "knows" that *the* is an article, *boy* and *pizza* are nouns, and *ate* is the past of *eat*, a verb. When the program recognizes *the* followed by a noun, it successfully matches this combination with a rule it has been given for forming noun phrases (NP → det N; i.e., "the noun phrase consists of a determiner and a noun"). The verb together with the other noun phrase (the pizza), matches the rule for the verb phrase (VP → V + NP). Finally, the noun phrase and the verb phrase together successfully match the rule for sentence (S → NP + VP), and the parse is complete.

The rules given here are simplified greatly for the sake of example. In actual practice, the biggest problem in programming such a parser is describing these rules correctly, i.e., the system's grammar. A simple phrase-structure grammar can deal with a fairly large subset of English sentences. For more complex sentences, parsers have used so-called "extended grammars" as in, for example, the augmented transition net grammar (ATN) used in SHRDLU, described in a later section. A full description of an extended grammar is beyond the scope of the present article; use of similar parsing programs in CALL software, however, is considered below.

Machine Translation

Some of the earliest work in AI attempted to develop translation programs that could somehow scan technical journals written in foreign languages and produce English versions automatically. The project turned out to be considerably more complicated than anticipated, and results were often absurd, if not unintelligible. One of the problems was that developers underestimated the need to parse the structure of the text they were dealing with. In an early paper, two pioneers in machine translation offered examples of word-for-word translations that their computer had produced

(Richens and Boothe, 15). These examples, according to the authors, "illustrate the relative unimportance of syntax . . . ":

Finnish: muut neljä ulkomaista kantaa ovat osoittautuneet viljelysarvoltaan kovin epävarmoiksi

English output: other [plural] four foreign country [out of] standpoint [partitive] bear [partitive] are show oneself [past plural] cultivation value [genitive accusative] very insecure [become]

The program has added lexical and structural clues to give the reader a more complete picture of the original. But when the English output is compared with a standard English equivalent, it is clear that syntax is anything but unimportant in understanding natural language:

Edited English: The other four standard varieties from abroad have proved reliable as regards cultivation value.

What such work did demonstrate was the feasibility of using computers to generate a first draft of a translation to speed up the work of the human translator who, in turn, could concentrate on editing and polishing the translation. Most "machine translation" is thus more accurately described, at least at present, as "machine-assisted translation."

Several machine-translation programs (both for mainframes and for micros) are now available commercially, with prices ranging from a few tens of thousands of dollars to ten times that number. Some produce so-called "high-quality" translations of scientific documents, i.e., with an accuracy of at least 90 percent, but depend heavily on pre- and/or postediting.

Current work on machine translation is centered on developing more sophisticated syntactic and semantic representations of language that can aid the program to understand more fully the material being translated. The Japanese Fifth Generation project has a goal of developing a system that would be perhaps 95 percent accurate and yet cost one-third less than human translators. Such a machine would need the help of an expert system—a set of programs that could represent a high level of different kinds of human knowledge, in this case syntactic, conceptual, and commonsense knowledge.

Speech Recognition

The full understanding of even written natural language requires the recognition of morphological, syntactic, and semantic elements. When the phonological information necessary to recognize speech is added, it is of little surprise that automated speech-recognition systems are still relatively primitive. We have not yet duplicated the virtuosity of Hal, the

on-board computer in *2001: A Space Odyssey* (Clarke, 4), who understood everything Dave said and could also detect overtones in his voice:

Hal: Look, Dave, you've got a lot of things to do. I suggest you leave this to me.
Dave: Hal, switch to manual hibernation control.
Hal: I can tell from your voice harmonics, Dave, that you're very upset. Why don't you take a stress pill and get some rest?

Speech recognition adds one more level of analysis to the list of steps described earlier for language understanding. At this initial level, the input is nothing more than sound waves, which first must be converted to waves of electrical current, then sampled thousands of times per second to determine their shape. The combined pattern of shapes gives an electrical "picture" of the sound: the picture in turn must be matched against a library of stored pictures in the system. When a match is found, the sound is recognized, and the system can report with some confidence, "That was a /p/".

In actual practice, however, much can go wrong. Human voices are all different, and no one pronounces the same word exactly the same way twice. In addition, strange things happen to sounds when combined. "Did you give the cat your dinner?" comes out like "Did you give the catcher dinner?" and so forth. Humans are helped by the context in which sounds are uttered and can make reasonably good guesses about what the speaker is trying to say. It appears that accurate speech recognition ultimately will have to use this same strategy: consult syntactic and semantic information to help the system determine what was probably said, given an assumed pragmatic environment.

Speech Synthesis

Speech synthesis is in many ways a simpler matter than speech recognition. The problem is one of simulating those same patterns of acoustic signals—i.e., phonemes—that humans recognize as part of the system of our language. There are now relatively inexpensive systems to accomplish this end. The user merely types an English sentence on the screen and the program voices it through its built-in speaker. A system now available for the IBM PC has an acceptable (if somewhat Swedish-sounding) accent in English (8). The so-called SMOOTHTALKER for the Apple Macintosh is not yet quite as elegant (18). The author has in his office a Spanish-speaking version of DECTalk, which pronounces typed-in sentences with any of a number of different voices, ranging from "Beautiful Betty" (soprano) to "Huge Harry" (baritone). The system, still in production, is being developed by the Digital Equipment Corporation for RENFE, the

Spanish national railroad, in order to synthesize constantly updated train information for those who call in.

Classic Experiments in Natural-Language Understanding

Computer scientists have been experimenting for at least twenty years with programs that can carry on a conversation with humans. Boden (3) describes early programs that could "talk" about baseball ("Who won the World Series in 1927?") or the weather ("What is the high today in Des Moines?"). The best-known are ELIZA (or DOCTOR) the sympathetic therapist, and SHRDLU, the expert in the world of toy blocks.

ELIZA

ELIZA (Weizenbaum, 23) was named after the woman in Shaw's "Pygmalion" who was taught to talk "right." Developed in 1966 at MIT, ELIZA can carry on a plausible conversation with a human typing at a terminal. The topic of conversation depends on the nature of ELIZA's "script," or programmed role. The most famous of these is DOCTOR, designed to simulate ("parody," says Weizenbaum) the role of a psychotherapist in a consultation by responding to the patient's "complaints" as below (ELIZA's responses are given in uppercase):

PLEASE TELL ME YOUR PROBLEM
I'm very unhappy these days
I AM SORRY TO HEAR YOU ARE UNHAPPY
The problem is my father
TELL ME MORE ABOUT YOUR FATHER
Last night I dreamt he spanked me
WHAT WOULD IT MEAN TO YOU IF HE SPANKED YOU?
Didn't you ever get spanked?
WE WERE TALKING ABOUT YOU, NOT ME

ELIZA creates the illusion of understanding through the use of a technique known as "pattern matching." For example, "I am X" is a pattern that causes the computer to respond with a prefabricated pattern of its own, such as "How long have you been X?" If X contains one of the words identified in the script as negative, such as *unhappy* or *depressed*, the response can then be "I am sorry to hear you are X." When the user types in something ELIZA doesn't recognize, the computer either hedges ("Why do you say that?" "Please go on," etc.) or it recycles a sentence the user typed earlier ("Earlier you mentioned that your brother has been picking on you"). Since ELIZA contains a large number of response patterns, the program is able to carry on a fairly lengthy dialog without repeating itself.

It should be obvious, however, that ELIZA has no real knowledge of what the user is saying. In fact, it is precisely because of this innocence that Weizenbaum chose to use the psychotherapist format: it is one of the few situations in human discourse in which, if the speaker mentions "boats," for example, one can naively respond, "What do boats mean to you?" or "Tell me more about boats." Another difficulty with ELIZA is that the program has very little knowledge of the structure of the sentences it receives, and can easily be misled. If the user happens to type a slightly more complex sentence such as "Thinking about what I am doing always upsets me," the program blindly applies the pattern " . . . I am X . . ." and responds with something like "How long have you been doing always upsets you?"

Nevertheless, ELIZA does manage to recreate much of the naturalness and spontaneity of real conversation. For this reason, a number of CALL researchers have experimented with foreign language versions of the program, which are described below.

SHRDLU

What is most glaringly missing in ELIZA, then, is any real knowledge about sentence structure or the subject at hand. The solution to the first problem is to build in a parser. The second is more complicated. One could either attempt to provide the program with an encyclopedia of knowledge about the world, or try to limit the topic of discussion to some micro-world it can handle. In a project known as SHRDLU, Winograd (24, 26) opted for the second approach. The micro-world in this case was limited to a set of a dozen or so toy blocks appearing on the screen. SHRDLU was programmed to understand everything that was typed in with respect to the position of these blocks, and could move them according to ordinary English instructions given by the person at the console. The following is part of a longer dialog with the system (Winograd, 25, p. 57); SHRDLU is in caps:

How many blocks are in the box?
FOUR OF THEM.
Is at least one of them narrower than the one which I told you to pick up?
YES, THE RED CUBE.
Can the table pick up blocks?
NO.
Can a pyramid be supported by a block?
YES.
Will you please stack up both of the red blocks and either a green cube or a pyramid?
OK. [Carries out command]
Which cube is sitting on the table?

THE LARGE GREEN ONE WHICH SUPPORTS THE RED PYRAMID.
Had you touched any pyramid before you put the green one on the little cube?
YES, THE GREEN ONE.
When did you pick it up?
WHILE I WAS STACKING UP THE RED CUBE, A LARGE RED BLOCK, AND A LARGE GREEN CUBE.
Why?
TO GET RID OF IT.
Why did you do that?
TO CLEAN OFF THE RED CUBE.

SHRDLU is not actually a single program, but a set of subprograms working together. Some of the subprograms are responsible for parsing the input sentences and determining how their parts are related. Others use the information from the parser to determine the meaning of the sentences. Still others are specialists at knowing about blocks and using this knowledge to reason about the intended meaning of the sentences. To do this, they must use information about the context, about the current state of the set of blocks. And they must have very specific knowledge about structural signals in English. For example, if instructed "Pick up a red block," the computer may pick up any of the red blocks; "Pick up the red block" signals that the speaker has a particular one in mind.

For all its sophistication, SHRDLU also has its limits—even within its own domain of expertise. As Winograd (27) points out, its grammar cannot handle phrases such as "three blocks high" or "a three-block stack." Nor can its reasoning program carry out one command while keeping track of another ("Build a stack without touching any pyramids"). And the program has only begun to address the problem of inference, of figuring out what the speaker really meant. Finally, Winograd cautions that expanding the program's world could create as many problems as it would solve. In an expanded world, the system would then have to face the problem of determining which information might be relevant to the question at hand; in the micro-world, everything is relevant.

Applications of AI to Language Learning

Computer-assisted language learning (CALL) offers a rich opportunity for employing many, if not all, of the techniques of natural-language processing in a useful and productive application. Yet it is curious that much of CALL exists in a world all its own. For the most part, CALL software shows little awareness of current thinking about how people learn language or how language teaching is best accomplished. Nor does CALL

seem to make use of the interesting techniques (parsing, semantic analysis, etc.) that have been developed in natural-language processing by computer. One could argue reasonably that the application of any technology in education should attempt to employ the best and most effective techniques currently known. In the case of CALL, this would mean considering, at least in part, artificial intelligence.

Natural-language systems can be developed to "understand" language on many different levels, from simple recognition of words to full syntactic and semantic parsing. Each of these concepts has been the subject of experimentation in the context of computer-assisted language instruction. One way of categorizing these efforts is in terms of the basic strategies they employ. Three groups define themselves: (1) conversational programs that employ keyword matching in the style of ELIZA, (2) programs that use syntactic parsing, with or without a knowledge base, and (3) intelligent or semi-intelligent adventure games, which generally incorporate some of the features of both (1) and (2).

Keyword Matching: ELIZA as Tutor

LIESL

Kossuth (9) describes her experiments with a German version of ELIZA, dubbed LIESL, adapted from an English version (in BASIC) published in a book of BASIC computer games (Ahl, 1). German faculty at Pomona College have developed ten different scripts for student use; topics of conversation range from discussion of stories in a reader used in class to everyday subjects such as the German class, weekend activities, etc. LIESL is updated every other week with a new script. The following is a sample of a conversation using the "Wochenende" (weekend) script:

TAG, LIESL HIER. WIE WAR DAS WOCHENENDE?
Super! (keyword: super)
TOLL, ERZÄHL MAL DAVON!
Ich war auf einer schönen Party. (Party)
WO WAR DIE PARTY? ICH HABE NICTS DAVON GEHÖRT.
Die Party war im Studentenheim. (Party)
HAST DU NETTE LEUTE KENNENGELERNT?
Nein, ich war mit meinem Freund da. (Freund)

Translation:
HI, THIS IS LIESL. HOW WAS YOUR WEEKEND?
Super!
TERRIFIC, TELL ME ABOUT IT!
I went to a great party.
WHERE WAS THE PARTY? I DIDN'T HEAR ABOUT IT.

The party was at a friend's house.
DID YOU MEET ANYONE NICE?
No, I went with my friend.

German instructors keep track of student use of these programs by requiring them to turn in a printout each week of a conversation with LIESL. Student use is aided by an on-line dictionary that students can call up for any word that LIESL uses by typing that word preceded by a question mark.

One advantage of an implementation such as LIESL is that it is built on a program in BASIC that can be adapted to run on virtually any microcomputer. One disadvantage of the BASIC version, however, is that it cannot offer the range of answer processing that is possible in the larger and more complex LISP version, such as the DOCTOR game available on most mainframe computers running the UNIX operating system. The principal strategy in the BASIC version is the matching of simple keywords, as in the example above. For each such keyword, there is a set of responses or response frames ("Why do you say that you are ____?"). The original LISP version, and versions adapted from it, allow the script to specify not only keywords, but different contexts in which these words may be found, thus determining more about what is being said. For example, different contexts could be given for the pronoun *you* (let X stand for whatever words might occur in that position):

you X me
I X you
I think you X
I want you to X

The response could thus be more exactly keyed to the input; for "I want you to X," ELIZA could respond with "Why do you want me to X," etc.

FAMILIA

Underwood (19, 20) describes an experimental simulated Spanish conversation program, FAMILIA, based on ELIZA. Adapted from the original LISP ELIZA, FAMILIA consists of two separate parts: (1) the original LISP language analyzer, which contains language-independent input routines for analyzing the user's sentences, and (2) a Spanish version of the script. In FAMILIA, the script was designed around the type of conversation one might have about one's family members, who they are, and what they are like.

One additional feature was added to the FAMILIA script—rules that allow the program to detect syntactic errors of certain types. Thus, prior to generating a response sentence, the program can check the syntax of the

input sentence for a recognizable error and type out an appropriate message. The key error FAMILIA anticipated was confusion in the use of the two verbs for "to be" in Spanish, *ser* and *estar*, since most of the input would consist of descriptive statements. Thus if the student volunteered, "My father is [está] very tall," the program could respond with "Do you mean your father is [es] tall?" and so forth. Evaluative feedback is accomplished by marking adjectives in the script as requiring one or other of the two verbs. FAMILIA also allows the student to ask questions about unfamiliar family terms the program might use, as in the following conversation (computer output is in upper case):

Hola
HOLA--VAMOS A HABLAR DE TU FAMILIA
Mi familia está en Colorado
¿ENTONCES UDS. SON DE COLORADO?
Bueno, mi madre está de Boston
¿QUIERES DECIR QUE TU MADRE ES DE BOSTON?
Mi madre es de Boston
¿Y TUS ABUELOS?
Mis abuelos están en Colorado también
¿TIENES TIOS?
No entiendo
¿SI USO UNA PALABRA QUE NO ENTIENDES, ESCRIBE: "QUE ES UN---?"
¿Qué es un tío?
¿TU TIO ES EL HERMANO DE UNO DE TUS PADRES
Sí, mi tía está profesora en Berkeley
¿QUIERES DECIR QUE TU TIA ES PROFESORA EN BERKELEY?
Mi tía es profesora
¿ALGUIEN MAS EN TU FAMILIA ES PROFESORA?

There is much to be said in favor of student practice with programs like LIESL and FAMILIA. They provide contextualized practice focused more on meaning than on form. The material is related to the student and hence interesting and motivating. And it is a form of comprehension-based practice, since the student cannot proceed without understanding what the program has said.

There are, however, obvious drawbacks. The main problem is simply that the program, for the most part, has no notion of the grammaticality or meaningfulness of the students' sentences, and looks merely for certain strings of letters (keywords) to which to respond with other strings of letters. The "conversation," then, is ultimately superficial. Although these programs allow for some amusing CALL activities, they can never know enough about what the student is doing to be engaged in any serious teaching. A further shortcoming in the keyword approach is that it works much better in a relatively uninflected language such as English since the

keyword can usually be listed in only one form. In contrast, in German or Spanish, keywords must have several different forms depending on tense inflections, gender, and so forth, each of which must be listed separately.

Parsing and Knowledge-Based Systems

In a review of different strategies for processing students' answers in CALL, Pusack (14) concludes that "the only real hope for truly effective foreign-language CAI" is through rule-based parsing (p. 63). Yet, as Pusack points out, to be successful such programs must be limited to a highly restricted domain, i.e., a small subset of the possible utterances in the language. It is unreasonable to expect a parser to handle all the sentences in a natural language; moreover, it simply is not necessary—the language that students can be expected to use in most CALL activities is limited to a relatively few lexical items and standard structures.

All the CALL programs that have incorporated some sort of parser successfully have in fact been quite limited in both structures and vocabulary. Four such programs, offered as examples of what can be done with parsing in the CALL environment, are considered below.

JOHN AND MARY

This experimental program (Higgins and Johns, 7) is part of a group of programs known as GRAMMARLAND, which the authors characterize as an attempt to use SHRDLU's principles for CALL "to create a miniature universe of discourse and a program that will manipulate things in that universe, answer questions about it, ask questions, or do any of these things at random if the user merely wants a demonstration" (p. 75). JOHN AND MARY creates a mini-world consisting of a room, a door, and two stick figures visible on the screen. The student manipulates this world with language, asking where the people are, or instructing the program to send them out or bring them in. The program can parse sentences such as the following:

Is Mary here?
Where is Mary?
Bring Mary in.
Send Mary out.
Shut the door.
Are John and Mary together?

The response to a question is a verbal answer; in the case of a command, there is a change in the graphics. Higgins and Johns give a complete

BASIC listing of a somewhat skeletal version of the program in the appendix to their book (7, pp. 177–87).

JUEGOS COMUNICATIVOS

JUEGOS COMUNICATIVOS, developed for publication in connection with a college Spanish textbook, consists of a set of problem-solving games designed for communicative practice in Spanish (Underwood and Bassein, 21; French and German versions are in production). JUEGOS actually consists of two rather separate components: a language-independent parsing program, and a set of language-specific data files in which word classes, morphological rules, and syntactic patterns are all specified for any particular environment. As noted above, the program's ability to parse intelligently depends on an environment limited to that of the mini-world displayed on the screen.

An example of one of the activities from JUEGOS is the game "Las cinco diferencias." Two almost identical pictures are displayed.

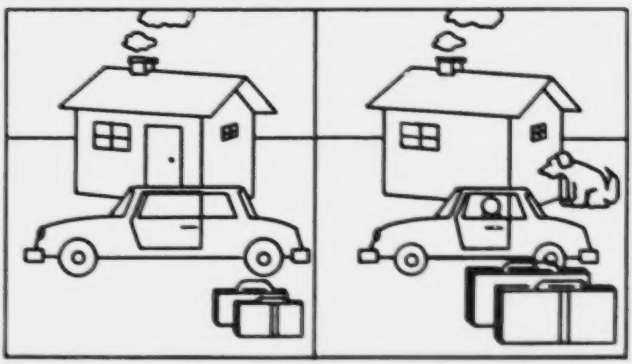


Fig. 1: On-screen Graphics for "Las cinco diferencias" (JUEGOS COMUNICATIVOS)

The student is instructed to describe the five things that are different in the picture on the right. With each input sentence, the program parses to see if it can recognize a structure. If so, the parser then checks the elements in that structure for problems of gender or concordance, missing elements, etc. When problems are detected, the questioned words are highlighted and some message is displayed to give the student a hint about the nature of the problem (¿Masculino o femenino? etc.). If no syntactic problems are found, the program checks the semantic content of the input sentence to see if it agrees with the picture. The following is a sample dialog about the picture above:

No hay puerta en la casa
TIENES RAZON
Las maletas son más grande
¿SINGULAR O PLURAL? [highlights "grande"]
Las maletas son más grandes
TIENES RAZON
Hay una ventana
¿COMO. . .?
Hay una ventana en la casa
SI, PERO NO ES UNA DIFERENCIA
Hay perro [highlights "perro"]
FALTA EL ARTICULO
Hay dos perros
YO NO VEO MAS QUE UNO

One reason the parser can be this accurate is that it does not have to attempt to handle vocabulary or structures beyond those needed to describe the picture on the screen. The few basic structures allowed are spelled out in the instructions (although many times there are half a dozen structural variations, all of which describe the picture correctly).

What enables the program to recognize these words and structures is the information given in the data file. This includes a lexicon in which root words are grouped by syntactic classes and subclasses (masculine nouns, feminine nouns, etc.). Morphological rules explain how plurals, etc., are formed from these roots (and hence recognized, by using the same rule to "peel off" the affix). Finally, rules are given for possible sentence patterns, in terms of syntactic units such as noun phrase (NP), prepositional phrase (PP), adjective phrase (AP), etc.; as a result, "Las cinco diferencias" requires only three basic patterns, with variations depending on choice of optional elements (indicated in parentheses):

(1)	(no)	hay	NP	(PP)	
(2)	NP	(no)	ser	AP	
(3)	NP	(no)	tener	NP	(PP)

It should be clear, however, that syntactic parsing is not enough, since it only allows the program to comment on the structure of the student's sentences, while remaining oblivious to their meaning. JUEGOS thus adds an element known as "semantic attributes," which amounts to declaring that there is a logical relationship between certain items that may be found in the input sentences. In the above picture, for example, there is a negative relationship between "house" and "door." This can be expressed in a number of syntactically different ways ("There is no door on the house," "The house doesn't have a door," etc.), which can be reduced to relating "house" and "door" in one of a number of possible negative structures. Since the program has first parsed for syntactic structure, it knows that these words occur together in the sentence; furthermore, it knows where

they occur, and that the sentence is negative. Based on that information, the program can safely assume that the correct assertion has or has not been made.

The suitcases in the picture above provide an example of this attribute relationship. In the semantic section of the lexical file, the following statement appears:

maletas = grandes largas
maletas ≠ pequeñas cortas.

This means there is a positive relationship between *maletas* and *grandes* or *largas*, and a negative relationship between *maletas* and *pequeñas* or *cortas*. Given this statement, all of the following sentences will be true:

Las maletas son grandes
Las maletas son largas
Las maletas no son pequeñas
Las maletas no son cortas

And, conversely, the following sentences will automatically be false:

Las maletas no son grandes
Las maletas no son largas
Las maletas son pequeñas
Las maletas son cortas

STATION

In a similar manner, Cook (5) describes programs in which syntactic parsing is complemented by some sort of knowledge representation that allows the program to "know" and recover essential information about the environment at hand. In an ESL program called STATION, students simulate dialogs with the local railway station in Colchester (England), inquiring when trains leave, which platform they leave from, how much the tickets cost, etc.:

HELLO. BRITISH RAIL, COLCHESTER.
Could you be telling me the train to Harwich?
SORRY, I DON'T UNDERSTAND
When is the next train to Harwich?
IT LEAVES AT THREE O'CLOCK
What platform is it?
IT LEAVES FROM PLATFORM 4
When does he arrive?
WHEN DOES WHAT?
When does it arrive?
IT ARRIVES AT HALF PAST THREE
How much is it?

IT COSTS TWO POUNDS.
Thank you. Goodbye.

As with the "semantic attributes" in JUEGOS, STATION stores information in terms of the relationship between items; here, this relationship is expressed in a manner that can be understood by a computer language known as PROLOG designed for the declaration of logical relationships:

leave(harwich,three)
ticket(harwich,two)
arrive(harwich,four)
platform(harwich,four)

"Leave(harwich,three)" means that there is a relationship called "leave" with two items "Harwich" and "three," that is, "The Harwich train leaves at three o'clock." If enough such information were programmed, the result would be in effect an expert system for students to consult in order to practice using language to accomplish one of its most important functions: get information.

A Computerized German Essay Processor

One application of parsing strategies is an aid to students who are writing sentences in a foreign language. The parser can help students with any errors that are detected to the extent that it can read these sentences and recognize their grammatical structure. Ruth and Alton Sanders (17) of Miami University (Oxford, Ohio) and Wright State University (Dayton, Ohio), respectively, have together received a major grant from the U.S. Department of Education to design a computerized essay processor for intermediate and advanced students of German.

The essay processor will be based on an expanded version of the parser used in the tutorial-game prototype, SPION (see below). Although the original parser was designed to recognize only WH-questions and command forms, and contains a lexicon limited to that needed for the game, the expanded version will be designed to parse all the basic syntactic structures of German (including word-order variants) and will contain a lexicon equivalent to the contents of a standard paperback German–English dictionary. In addition to syntactic parsing, the program will check for both spelling problems and morphological anomalies (wrong case, wrong gender, etc.) in student input.

The authors do not anticipate creating an infallible system, but one that will provide the student with a useful and comprehensive check on the material written. The program will not attempt to correct errors, but will simply flag them; the authors believe it is more useful (and safer) pedagogically to help the students find errors than to try to help edit them.

Given the wide range of vocabulary and structures the essay processor

will accept, it was considered unrealistic to attempt to include a semantic component. As a result, the program will be purposefully oblivious to the meaning of what the student has written, and will be unable to detect semantic errors of the "Sincerity admires John" type. Students who intentionally write nonsense to fool the system (as students tend to do) will have to be content with having the system help them write grammatically correct nonsense.

The essay processor is being developed in LISP and PROLOG, and uses an augmented phrase-structure grammar for syntactic parsing. When completed, it will be implemented in IBM PC format and will be made available without charge for field testing to interested college and university foreign language departments.

Intelligent Games

There are many computer adventure games with which the reader may be familiar (not video games of the joystick variety, but those that require the player to type in responses). Most operate on the same principle: The computer describes the situation ("There is an open door on your right, stairs on your left," etc.), and asks what the player wants to do next. In an intelligent game, this is an open choice, not just "pick one of the following." Allowing the player to input a somewhat free response means, of course, that there must be at least a certain amount of parsing involved in order to find out what has been typed in. In actual practice, this parsing is usually fairly primitive, often allowing the player to respond with nothing more than "Open door" or even, in some cases "door open" (in the latter case, the "parser" is actually only looking for a word match, and ignores word order).

ZORK

The best-selling game, ZORK, a product of MIT graduate students of the 1970s, is available in commercial versions (28) and as part of mainframe operating systems such as UNIX. The following represents the opening dialogue:

ZORK: You are standing in an open field west of a white house with a boarded front door. There is a small mailbox here.
User: Look in the mailbox.
ZORK: The mailbox is closed.
User: Open it.
ZORK: Opening the mailbox reveals a leaflet.
User: Read the leaflet.
ZORK: "Welcome to ZORK . . . ."

The interchange that takes place when using such a game can be genuinely meaningful, an example of true reading for meaning, since "the user does not think about the language in use, but only about the action and where it might lead next . . . . [This] creates an optimal setting for language acquisition" (Kossuth, 10, p. 13). And the natural motivation to play games means that students will use them in earnest and will repeat adventures in order to find a better way to "beat the game." Because of the multiple branching possibilities of such programs, the user will never read through the same sequence twice. Student users, Kossuth claims, "voluntarily read more than they would if assigned a linear text, and their comprehension can be expected to increase with each repetition" (ibid).

MYSTERY HOUSE

Any adventure game of the type just described can be improved with the addition of computer graphics so that the scene can actually be viewed by the user. One game that does in fact include pictures is MYSTERY HOUSE (13), a program in which a series of murders must be solved. As the scenes unfold, the graphics change. When the user instructs the program to open the door, the picture of a closed door is replaced by one of an open door. The addition of relevant visual material adds to the screen an important element of redundancy, with the result that the language used is potentially more meaningful to the user.

The French version of this program, however, is a classic example of doing things in a hurry. Intended for language practice rather than game-playing, the program has serious flaws in design, the most glaring of which is a lack of sensitivity to the structure of French. For example, not only does it accept "ouvrez porte" in place of "ouvrez la porte," the language generator that produces responses, alas, comes back frequently with ungrammatical French. The original strategy for dealing with unrecognized commands was to insert the input verb into a canned response frame: "I don't know how to (verb)." The strategy is a safe one in English, for all command forms are the same as infinitive forms. But command forms and infinitives in French are always different. The "solution" in the French version of MYSTERY HOUSE is to convert all input commands to one form of the infinitive (and even then to the least common, the one ending in -re). The result is some very disconcerting French: "*Je ne sais pas cherchre," instead of "Je ne sais pas chercher," etc.

This "goof" is given as an example of artificial intelligence gone bad. An adventure game in which student input must be "understood" and responded to in a natural communicative fashion is clearly an excellent opportunity for using at least some natural-language processing techniques. The French version of MYSTERY HOUSE, then, can best be described as an unfortunate "occasion manquée."

SPION

SPION (Sanders and Sanders, 16) is a German spy game that takes the student through a world of German airports and hotels. Unlike the two games described above, SPION was written with language learning in mind and therefore attempts to provide a somewhat more useful set of situations than haunted houses and dungeons. SPION also incorporates a certain number of on-line aids for the student.

SPION allows the student to respond with either commands or Wh-questions. If students are puzzled over a word the program has used, they can instead type in "?", which takes them to a dictionary containing most of the vocabulary used in the program. Responses to situations are given as a choice: "Should I read the note, take the plane, go into the café, take the bus . . .?" (which simplifies the parsing problem when the student types a choice). The program also insists that each German sentence be spelled correctly and that the words be in the right order. If the input is lexically and grammatically correct it is sent to the "semantic interpreter" to check which of the possible choices the student has intended. If a grammatical problem is detected, as when a student uses an incorrect verb form, a generic message is typed out:

User: Gene . . . [for Gehen, "Go . . . "]
SPION: Das scheint mir nicht richtig. Versüchen Sie noch einmal! ["That doesn't look right to me. Please try again!"]

The feedback, although not specific to the error, signals to the student that there is a problem with the form of the statement. Sanders and Sanders claim to have avoided more analytical error feedback deliberately, mainly so that the program will be perceived more as a game than as a language exercise, and because they believe it is not helpful to tell students exactly what is wrong. As in the case of JUEGOS, however, it is clearly possible to give feedback that hints at error and still forces the student to think about and discover where the problem lies.

Current Major Projects Using AI in CALL

Project Athena

Potentially the most interesting project in which AI strategies are being applied directly to CALL is Project Athena, currently underway in the Foreign Languages and Literatures Section at MIT. Part of a five-year campuswide experiment in integrating computers into the curriculum, the MIT/CALL project makes full use of one of the best AI facilities in the world. The goal is to develop materials for teaching a four-course sequence in French, Spanish, Russian, German, and ESL. The materials are

intended to supplement but not replace classroom time or the teacher; they will, however, replace both the textbook and the audio lab.

Part of the Athena project will incorporate interactive video. Of more interest here are the plans for the natural-language processing of typed-in sentences. This component of the system will make use of a parser connected to a system for knowledge representation that will enable the computer to understand the meaning of the sentences. The program will thus be able to detect the student's semantically incoherent or culturally inappropriate statements, in addition to flagging spelling mistakes, typos, and grammatical errors.

As of this writing, none of the work on Project Athena has been demonstrated or described in any detail. However, published descriptions of planned programs are found in Kramsch, Morgenstern, and Murray (11), and include three distinct programs: LINGO, Topicks, and what is tentatively called Reverse Eliza (named for the ELIZA discussed above). One of these, LINGO, is described as follows:

> A student is shown a picture of a dorm room with some representative German objects in it and is introduced to a poltergeist with whom the student is invited to converse. The poltergeist will mess up the room following the student's orders, but in order to talk with the spirit the student must follow certain rules of conversation. Our parser and knowledge representation system are powerful enough so that we can accept sentences of much greater complexity than the "kill troll with sword" variety . . . . The messed up room picture will be stored to be retrieved by another student who will be given the task of cleaning up the room (p. 32).

The student learns to interact with the ghost in a manner resembling natural discourse. For example, the computer may not understand the student's directions and may ask for clarification, or it may suggest ways in which to improve the language being used.

"Reverse ELIZA" is a curious variation on the "standard" ELIZA. Instead of answering ELIZA's questions or comments, the student must obtain information from a rather terse and laconic conversation partner, one who knows the answers but must in effect be coaxed to give them up. The point is to learn those communicative skills necessary to "pry information out of a reluctant informant, using tactful questioning techniques" (ibid). If standard Wh- questions are used, Reverse ELIZA will give only scanty information; the user will find it more successful to employ indirect approaches, such as "reactions (Oh, really?), commentaries (That's too bad!), repetitions and paraphrases (You were sick?), interpretations and inferences (So you mean you had to stay at home)." The student thus learns very much the same strategies the original ELIZA (as DOCTOR) employed to keep the conversation going.

University of Rochester's TALK Project

A team of language teachers and programmers in AI at the University of Rochester are developing a parser-based authoring system for CALL materials (2). Called TALK (Teacher's Authoring Language Kit), the system will allow instructors to produce materials with built-in error-analysis capabilities. Rather than match answers literally as strings of characters (in the manner of authoring systems such as DASHER or EnBASIC), TALK will parse the student's input and check for syntactic category, morphological problems, etc. The program is being developed in a combination of LISP and the C programming language.

A unique feature of TALK will be the ability of the system to actually talk to the student by reproducing segments of stored speech at appropriate places in the program. "Speaking to the student" will be accomplished by means of digitized rather than synthesized speech (human voice segments recorded and stored in digital form—in a manner similar to compact disk recordings for music—on the same data disk that contains the program information), an arrangement that has the advantage of providing high-quality authentic speech segments that can be accessed by the program at random when needed. The TALK system will be designed so that the author using it can easily include commands for playback of these segments in the instructional program.

Project Alexis (Mills College)

The Foreign Language Department at Mills College (California) has begun initial planning and development of Alexis, a multifunction language-learning system for the VAX computer. Alexis is intended for first- and second-year students in French, German, and Spanish who will have access to VAX terminals located at various points on campus, including dormitories and the language lab. A unique feature of the system is the variety of activities it offers the student: exercises in reading comprehension, grammar (featuring an "intelligent" authoring system), compositions mailed to the instructor through foreign-language electronic mail systems, conversation (different versions of ELIZA), and games.

All activities in the Alexis system will also have access to several help features, including a full "helpscreen" to allow the student to move to any point in the system, complete on-line instructions, and on-line dictionaries for all three languages. When complete, the latter will contain the full first- and second-year lexicon corresponding to the contents of textbooks used at that level; definitions will be given in the foreign language.

Development work on Alexis is quite advanced, in part due to the strategy of the team charged with developing software. Wherever possible, the

system incorporates existing programs that can be adapted to the purposes of the project. The reading program, for example, is an exercise developed by a Mills graduate student as a thesis project; the grammar authoring system is being developed by a computer-science professor as a by-product of another project; the electronic mail was already in place; the ELIZA program, and one of the games (ZORK), is available to UNIX users as public-domain software on most versions of the operating system; the remaining games were developed by two of the team members on contract to Random House Publishers, Inc., who have given permission for the incorporation of their product into this noncommercial system. When complete, Alexis promises to be a model for using mainframe academic computers (of which the VAX is the most common) for language practice.

Intelligent Grammar Tutor (University of Pittsburgh)

Vernick and Levin (22) describe work in progress in the design of an intelligent grammar tutoring system for ESL. Their instructional model extends the notion of an "intelligent" CALL system to include three different kinds of information they feel the system should possess: (1) knowledge of the subject matter (i.e., the lexicon and grammar of the items in the exercise), (2) knowledge the system acquires about the types of errors the student makes, and (3) knowledge of the most appropriate pedagogical strategies for the handling of specific types of errors. The addition of components (2) and (3) will enable the system to decide on the kinds of feedback to provide the student, as well as how much and when to provide it. Taking into account the learner's past and present performance, as well as the types of errors made, this feedback may consist of advice to the student in the form of suggestions for further practice and testing.

The ESL system described by Vernick and Levin will also possess intelligence of the type described in this chapter: knowledge of the structure of the language, more specifically that part of the language being practiced. In this case the structures involved include the passive and relative clauses in English. Using a limited lexicon and a set of grammar rules, the parsing component will compare student input with the instructor's model and note any morphological or syntactic problems. (Spelling, since it is not the point of the exercise, will be corrected automatically by a routine that compares misspelled words with a list of correctly spelled words.)

Getting Started in Intelligent CALL

Much of what is described above is clearly beyond the reach of the average language teacher who has an Apple IIe and a little knowledge of BASIC.

Some of the techniques described herein may be incorporated only partially into one's CALL efforts; perhaps they suggest no more than a basic approach—an attitude toward using, or trying to use, computers in a better way.

The more ambitious may wish to explore ways to try out some of the techniques described. As should be clear from the description of the less complex programs (e.g., SPION), there is much that can be accomplished even on a small scale. This section is devoted to outlining ways to experiment with building better mousetraps.

The artificial intelligentsia's favorite programming language is LISP. Although it is a fascinating language to puzzle through, a LISP program is usually far from transparent. LISP functions, for example, can be somewhat cryptic: (setq 'verb (car (cdr sentence)) is LISP for "let 'verb' be equal to the first element [= car] of everything but the first element [= cdr] of 'sentence'."

A more accessible language, and one undoubtedly more available to users of the typical school microcomputer, is LOGO. Although popular in schools for the teaching of geometrical reasoning through what is known as "turtle graphics" (making an object called the turtle move on the screen according to student instructions), LOGO, like LISP, is actually a powerful string-handling language, i.e., a language for manipulating words and sentences. Most versions of LOGO can be adapted to the kind of natural-language techniques that go into a good interactive program. And LOGO commands tend to be considerably more mnemonic and straightforward; the instruction given above in LISP would in LOGO read something like "MAKE 'VERB [FIRST BUTFIRST SENTENCE]."

How does one begin? As with everything else, start with something small. Play with writing LOGO routines that manipulate sentences or parts of sentences successfully. Most LOGO manuals give examples of short language programs or routines, such as a procedure for converting normal English sentences to Pig Latin ("igpay atinlay"). Most users have found that trying out such programs gives them ideas for going on to something more relevant to their interests. The teacher/programmer is limited only by the boundaries of his or her creativity.

One alternative to this do-it-yourself approach is to purchase either a ready-made adventure game or one of the new adventure-game authoring systems now available, and adapt to it to one's one language and tastes. Kossuth (10) describes such systems and her experiments with adapting such programs. The results might be described as instant AI. This author's experimentation with ELIZA similarly amounted to taking an existing (and complicated) program, translating it into Spanish, and reworking it to meet his own structural and pedagogical goals (19, 20).

Conclusions

What We Know

When one takes a close look at what natural-language processing can (and cannot) do, one gets a better sense of both the limitations in, and the possibilities for, developing a more "intelligent" CALL. It is clear that full language understanding by computer is not within the range of current possibilities. One must recognize, as did Weizenbaum and Winograd, that even the most clever of programs does not approach the understanding of the human speaker in normal language use; says Winograd (25) of SHRDLU, "the heuristics in our system touch only the tiniest bit of the relevant knowledge" (p. 191).

Yet it is equally clear that it is unnecessary for a CALL system to achieve anything approaching full understanding in order to be useful. Even the "semi-intelligent" system, endowed with partial understanding of language structure and/or meaning, can provide a rich and meaningful form of interaction with the student. A grammar exercise can use parsing strategies to provide a much more specific form of feedback. A game program can be programmed with certain simple semantic rules that allow it to judge the appropriateness of the student's input for the situation at hand. In each instance the success of the program will depend on its having a carefully restricted domain, i.e., a limited range of lexicon and structures it must "understand."

What the Future Holds

It is probably much too early to try to speculate where this experimentation will lead, particularly since artificial intelligence itself is, some say, still in the horse-and-buggy stage. But it seems safe to say that CALL is beginning to move in a much more interesting direction, and that perhaps the emphasis on drill-and-practice programs will shortly become a thing of the past. As with any new technology, the language lab included, initial efforts tend to copy the kinds of things one has been used to doing. CALL initially looked suspiciously like "programmed instruction," a logical, if unfortunate, model for using what appeared at first to be yet another "teaching machine." But it should be clear that today's computers are a far cry from mere teaching machines. What they can do is virtually endless, bounded not so much by the mechanical limits of the machines themselves as by our own imagination.

References: Artificial Intelligence and CALL

1. Ahl, David H. *More BASIC Computer Games*. New York: Workman, 1979.

2. Blake, Robert. "CALL and the Language Lab of the Future." Paper in preparation, 1986.
3. Boden, Margaret A. *Artificial Intelligence and Natural Man*. New York: Basic Books, 1977.
4. Clarke, Arthur C. *2001: A Space Odyssey*. New York: New American Library, 1968.
5. Cook, Vivian. "Designing CALL Programs for Communicative Teaching." Paper in preparation, 1986.
6. Feigenbaum, Edward A., and Pamela McCorduck. *The Fifth Generation: Artificial Intelligence and Japan's Computer Challenge to the World*. New York: New American Library, 1984.
7. Higgins, John, and Tim Johns. *Computers in Language Learning*. Reading, MA: Addison-Wesley, 1984.
8. IBM PC Voice Communications Option. White Plains, NY: IBM, 1985.
9. Kossuth, Karen C. "Suggestions for Comprehension-Based Computer-Assisted Instruction in German." *Die Unterrichtspraxis*, 17, 1 (1984):109–15.
10. _____. "Using the Adventure Formats for CALL." *CALICO Journal* 3, 2 (1985):13–17.
11. Kramsch, Claire, Douglas Morgenstern, and Janet H. Murray. "An Overview of the MIT Athena Language Learning Project." *CALICO Journal* 2, 4 (1985):31–34.
12. Marbach, William D. "The Smarter Computer." *Newsweek*, 3 December 1984, pp. 89–91b.
13. Mystery House. Program for the Apple II Computer. Coarsegold, CA:Sierra On-Line Systems, 1980.
14. Pusack, James P. "Answer-Processing and Error Correction in Foreign Language CAI." *System* 11, 1 (1983):53–64.
15. Richens, R. H., and A. D. Booths, eds. *Machine Translation of Languages*, New York: Wiley, 1955.
16. Sanders, Ruth H., and Alton F. Sanders. "'Intelligent' Games for German Language Teaching," pp. 141–46 in *Foreign Language Instructional Technology Conference Proceedings, 21–24 September, 1982*. Monterey, CA: Defense Language Institute, 1983.
17. Sanders, Ruth H., and Alton F. Sanders. A German essay processor, personal communication.
18. SMOOTHTALKER Speech System. Program for the Apple Macintosh Computer. Long Beach, CA: First Byte, Inc., 1985.
19. Underwood, John. "Simulated Conversation as a CAI Strategy." *Foreign Language Annals* 13 (1982):209–12.
20. _____. *Linguistics, Computers, and the Language Teacher: A Communicative Approach*. Rowley, MA:Newbury House, 1984.
21. _____, and Richard Bassein. Juegos Comunicativos: Spanish Games for Communicative Practice, Program for the Apple IIe Computer. New York: Random House, 1985.
22. Vernuil, Judy, and Lori Levin. "Intelligent Grammar Tutoring in ESL." Paper presented at the 1986 TESOL Convention, Anaheim, California, March 1986.
23. Weizenbaum, Joseph. "ELIZA—A Computer Program for the Study of Natural Language Communication between Man and Machine." *Communications of the Association for Computing Machines*, 10 8(1967):474–80.
24. Winograd, Terry. *Understanding Natural Language*. New York: Academic Press, 1972.
25. _____. "A Procedural Model of Language Understanding," pp. 154–86 in

Roger Schank and Kenneth Colby, eds., *Computer Models of Thought and Language*. San Francisco: W. H. Freeman, 1973.

26. _____ "Towards Convivial Computing," pp. 56–72 in Michael J. Dertouzos and Joel Moses, eds., *The Computer Age: A Twenty-Year View*. Cambridge, MA: MIT Press, 1979.
27. _____ "Computer Software for Working with Language." *Scientific American*, 251,4(September 1984):131–45.
28. ZORK. Program for the IBM PC Computer. Cambridge, MA: Infocom, 1982.